```
loadUserData = makeLoadData 'user'
saveUserData = makeSaveData 'user'
```

You can use the `makeLoadData` function to create a function for `loadAnythingData`, literally:

```
loadAnythingData = makeLoadData 'anything'
```

The `makeLoadData` and `makeSaveData` functions are abstractions that allow you to create individual functions that are useful in specific circumstances. Now look at `makeLoadData` and `makeSaveData`; they're basically the same. Extract the common parts to a single function:

```
makeDbOperator = (db) ->
  (operation) ->
    (entry, value=null, callback) ->
      db[operation] entry, value, (error, data) ->
        callback? error, data
```

This function returns a function that returns a function. When you see an abstraction by itself, it becomes apparent if it's more complicated than it needs to be. You revise `makeDbOperator` to make it simpler:

```
makeDbOperator = (db) ->
  (operation) ->
    (entry, params...) ->
      db[operation] entry, params...
```

When you added global variable assignments to your program, you got an immediate payoff, but it hurt you in the long run. Identifying and writing abstractions is the opposite. A little bit of effort now to create the right abstraction saves you pain in the long run. You can now generate `load` and `save` functions for different data types from this single `makeDbOperator` function:

```
loadProductData = (makeDbOperator 'product') 'get'
saveProductData = (makeDbOperator 'product') 'set'

saveProductData 'photonify1100', 'data for the photonify1100'
loadProductData 'photonify1100'
# 'data for the photonify1100'
```